Part 1: Multiple Choice Questions (10 MCQs)

Question 1: DDL vs. DML - TRUNCATE vs. DELETE Which statement correctly differentiates TRUNCATE TABLE from DELETE FROM table_name in MySQL?

Ans:

B) TRUNCATE is a DDL command that removes data by deallocating data pages without firing individual row-level triggers, whereas DELETE is a DML command that deletes records row-by-row and records each deletion in the transaction log.

Question 2: Foreign Key Constraints & Cascading Actions Given a table Orders with a Foreign Key referencing Customers(customer_id) ON DELETE SET NULL, what happens when a record in Customers with customer_id = 5 is deleted?

Ans:

C) Customer 5 is deleted, and the customer_id column in Orders for matching rows is set to NULL.

Question 3: Emulating FULL OUTER JOIN in MySQL MySQL does not natively support the FULL OUTER JOIN keyword. Which combination of SQL constructs produces the exact functional equivalent of a FULL OUTER JOIN between Table A and Table B?

Ans:

B) SELECT * FROM TableA LEFT JOIN TableB ON TableA.id = TableB.id UNION SELECT * FROM TableA RIGHT JOIN TableB ON TableA.id = TableB.id;

Question 4: Subqueries - EXISTS vs. IN Operator Consider the query: SELECT * FROM Customers c WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.customer_id = c.customer_id); How does MySQL process this correlated subquery using EXISTS?

Ans:

B) It evaluates the subquery for each row of the outer table and terminates the inner scan as soon as the first matching order record is found.

Question 5: Database Normalization - 3NF Definition A database table is in Third Normal Form (3NF) if and only if it satisfies which set of conditions?

Ans:

B) It is in 2NF and every non-key column is directly dependent on the primary key (no transitive dependencies).

Question 6: DQL Execution Order - WHERE vs. HAVING In a DQL statement containing WHERE, GROUP BY, and HAVING clauses, in what order does the MySQL execution engine evaluate these clauses?

Ans:

C) WHERE → GROUP BY → HAVING

Question 7: MySQL Constraints - UNIQUE Key with NULL Values In MySQL (InnoDB engine), how does a column defined with a UNIQUE constraint handle NULL values?

Ans:

B) It allows multiple NULL entries because NULL represents an unknown value and is not considered equal to another NULL.

Question 8: DDL ALTER TABLE Operations Which command adds a new column named birth_date of type DATE with a NOT NULL constraint to an existing table named Customers?

Ans:

B) ALTER TABLE Customers ADD birth_date DATE NOT NULL;

Question 9: Self Join Concepts Which business scenario requires a Self Join (joining a table to itself)?

Ans:

B) Finding employees and the names of their direct managers when both employees and managers are stored in the same Employees table.

Question 10: DML UPDATE with Subqueries What happens if you run the following MySQL query? UPDATE Orders SET amount = amount * 1.05 WHERE customer_id = (SELECT customer_id FROM Customers WHERE country = 'USA'); (Assume there are 3 customers residing in 'USA')

Ans:

B) MySQL will throw an error because the single-row comparison operator = is used with a subquery that returns multiple rows.

Part 2: Hands-On Practical SQL Tasks

Task 1: Multi-Table DQL Aggregation with INNER JOIN

Objective: Calculate the total amount spent by each customer across all their orders. Display first_name, last_name, and total_spent. Sort the results by total_spent in descending order.

Input

Run SQL

```
-- Task 1: Multi-Table DQL Aggregation with INNER JOIN
SELECT
    c.first_name,
    c.last_name,
    SUM(o.amount) AS total_spent
FROM Customers c
INNER JOIN Orders o
    ON c.customer_id = o.customer_id
GROUP BY
    c.customer_id,
    c.first_name,
    c.last_name
ORDER BY total_spent DESC;
```

Output

first_name	last_name	total_spent
David	Robinson	12000
John	Reinhardt	700
John	Doe	400
Robert	Luna	250

Task 2: Unmatched Records Identification using LEFT JOIN

Objective: Identify all customers who have never placed an order. Display their first_name, last_name, and country.

Input

```sql
-- Task 2: Unmatched Records Identification using LEFT JOIN
SELECT
c.first_name,
c.last_name,
c.country
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Output

first_name	last_name	country
Betty	Doe	UAE

Task 3: Filtering via Scalar Subquery

Objective: Retrieve details of all orders where the order amount is strictly greater than the overall average order amount across the database. Display order_id, item, amount, and customer_id.

Input Run SQL

```sql
-- Task 3: Filtering via Scalar Subquery
SELECT
    order_id,
    item,
    amount,
    customer_id
FROM Orders
WHERE amount > (
    SELECT AVG(amount)
    FROM Orders
);
```

Output

order_id	item	amount	customer_id
3	Monitor	12000	3

Task 4: Correlated Subquery using EXISTS

Objective: Retrieve the first_name and last_name of customers who have at least one shipping record with a status of 'Pending' in the Shippings table.

Input Run SQL

```sql
-- Task 4: Correlated Subquery using EXISTS
SELECT
    c.first_name,
    c.last_name
FROM Customers c
WHERE EXISTS (
    SELECT 1
    FROM Shippings s
    WHERE s.customer = c.customer_id
      AND s.status = 'Pending'
);
```

Output

first_name	last_name
Robert	Luna
John	Reinhardt
Betty	Doe

Task 5: Grouped Filtering using GROUP BY & HAVING

Objective: Display each country along with the average customer age for customers in that country. Filter the results to include only countries that have more than 1 customer and where the average age

is greater than 20.

Input

```
-- Task 5: Grouped Filtering using GROUP BY & HAVING
SELECT
country,
AVG(age) AS avg_age
FROM Customers
GROUP BY country
HAVING COUNT(customer_id) > 1
AND AVG(age) > 20;
```

Output

country	avg_age
UK	23.5
USA	26.5

Task 6: DDL Table Creation with Constraints (Manual Primary Key)

Objective: Write a DDL statement to create a table named VIP_Memberships without using AUTO_INCREMENT: ● vip_id: Integer, Primary Key. ● customer_id: Integer, Foreign Key referencing Customers(customer_id) with cascading deletes. ● tier: String up to 50 characters, defaulting to 'Gold'. ● discount_rate: Decimal (4,2) with a CHECK constraint ensuring values are between 0.00 and 0.30.

Input Run SQL

```
-- Task 6: DDL Table Creation with Constraints (Manual Primary Key)
CREATE TABLE VIP_Membership
 (
  vip_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  tier VARCHAR(50) DEFAULT 'Gold',
  discount_rate DECIMAL(4,2)
  CHECK (discount_rate >= 0.00 AND discount_rate <= 0.30),
  FOREIGN KEY (customer_id)
  REFERENCES Customers(customer_id)
  ON DELETE CASCADE
 );
```

Output

SQL query successfully executed. However, the result set is empty.

VIP_Membership

vip_id	customer_id	tier	discount_rate
empty			

Task 7: Dynamic DML Insertion without AUTO_INCREMENT

Objective: Write a DML statement to insert a new order for customer 'John Doe' (item = 'Webcam', amount = 150). Since order_id is a primary key without AUTO_INCREMENT, generate the next order_id dynamically as COALESCE(MAX(order_id), 0) + 1 alongside the subquery for customer_id.

Input

Run SQL

Available Tables

```
INSERT INTO Orders (order_id, item, amount, customer_id)
SELECT
COALESCE(MAX(order_id), 0) + 1,
'Webcam',
150,
(SELECT customer_id FROM Customers WHERE first_name = 'John' AND last_name =
'Doe' LIMIT 1)
FROM Orders;
```

Output

SQL query successfully executed. However, the result set is empty.

Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3
4	Keyboard	400	1
5	Mousepad	250	2
6	Webcam	150	1
7	Webcam	150	1

Task 8: Conditional DML Update with Subquery

Objective: Write a DML query to increase the amount by 10% for all orders placed by customers residing in the 'USA'.

Input

Run SQL

Available Tables

```
-- Task 8: Conditional DML Update with Subquery
UPDATE Orders
SET amount = amount * 1.10
WHERE customer_id IN (
SELECT customer_id
FROM Customers
WHERE country = 'USA'
);
```

Output

SQL query successfully executed. However, the result set is empty.

Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3
4	Keyboard	440.00000000000006	1
5	Mousepad	275	2
6	Webcam	165	1
7	Webcam	165	1

Task 9: Multi-Table Joins (3 Tables)

Objective: Construct a query joining Customers, Orders, and Shippings to display first_name, last_name, order item, order amount, and shipping status. Ensure orders are displayed even if their shipping record is unassigned.

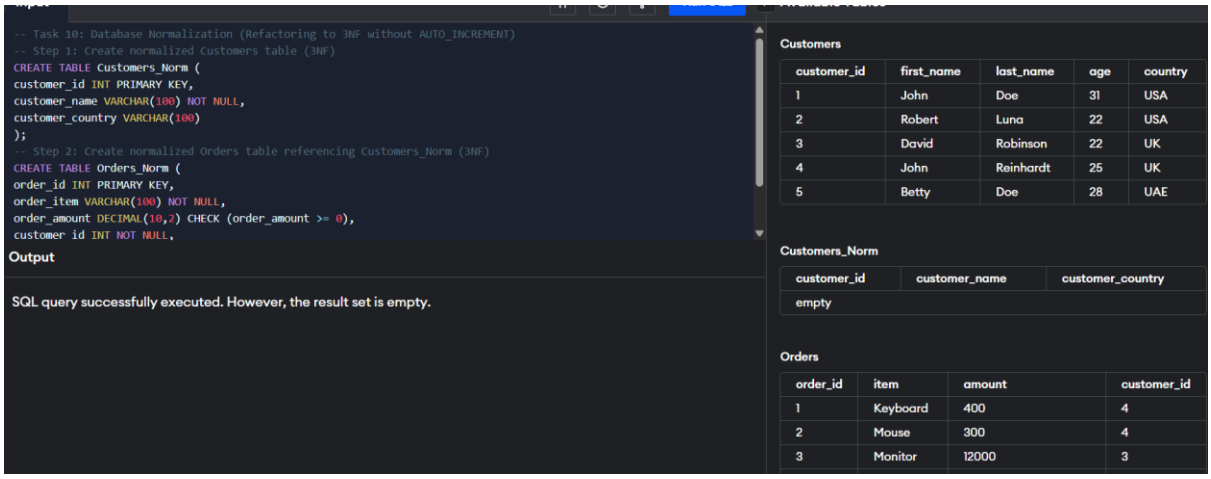
```
-- Task 9: Multi-Table Joins (3 Tables)
SELECT
c.first_name,
c.last_name,
o.item,
o.amount,
COALESCE(s.status, 'Not Shipped') AS shipping_status
FROM Customers c
INNER JOIN Orders o ON c.customer_id = o.customer_id
LEFT JOIN Shippings s ON c.customer_id = s.customer;
```

Output

first_name	last_name	item	amount	shipping_status
John	Reinhardt	Keyboard	400	Pending
John	Reinhardt	Mouse	300	Pending
David	Robinson	Monitor	12000	Delivered
John	Doe	Keyboard	440.00000000000006	Delivered
Robert	Luna	Mousepad	275	Pending
John	Doe	Webcam	165	Delivered
John	Doe	Webcam	165	Delivered

Task 10: Database Normalization (Refactoring to 3NF without AUTO_INCREMENT)

Objective: Convert the following unnormalized table into Third Normal Form (3NF) by providing DDL statements using explicit Primary Keys. Unnormalized Table (Unnormalized_Sales): customer_id, customer_name, customer_country, order_id, order_item, order_amount



```
-- Task 10: Database Normalization (Refactoring to 3NF without AUTO_INCREMENT)
-- Step 1: Create normalized Customers table (3NF)
CREATE TABLE Customers_Norm (
customer_id INT PRIMARY KEY,
customer_name VARCHAR(100) NOT NULL,
customer_country VARCHAR(100)
);
-- Step 2: Create normalized Orders table referencing Customers_Norm (3NF)
CREATE TABLE Orders_Norm (
order_id INT PRIMARY KEY,
order_item VARCHAR(100) NOT NULL,
order_amount DECIMAL(10,2) CHECK (order_amount >= 0),
customer id INT NOT NULL,
```

Output

SQL query successfully executed. However, the result set is empty.

Customers

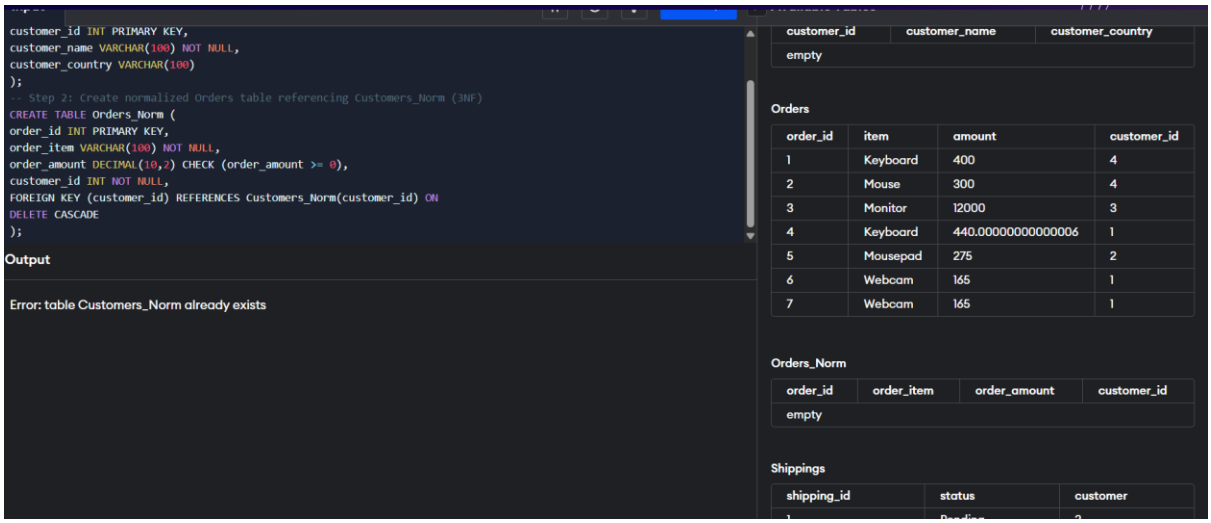
customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

Customers_Norm

customer_id	customer_name	customer_country
empty		

Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3



```
customer_id INT PRIMARY KEY,
customer_name VARCHAR(100) NOT NULL,
customer_country VARCHAR(100)
);
-- Step 2: Create normalized Orders table referencing Customers_Norm (3NF)
CREATE TABLE Orders_Norm (
order_id INT PRIMARY KEY,
order_item VARCHAR(100) NOT NULL,
order_amount DECIMAL(10,2) CHECK (order_amount >= 0),
customer_id INT NOT NULL,
FOREIGN KEY (customer_id) REFERENCES Customers_Norm(customer_id) ON
DELETE CASCADE
);
```

Output

Error: table Customers_Norm already exists

customer_id	customer_name	customer_country
empty		

Orders

order_id	item	amount	customer_id
1	Keyboard	400	4
2	Mouse	300	4
3	Monitor	12000	3
4	Keyboard	440.00000000000006	1
5	Mousepad	275	2
6	Webcam	165	1
7	Webcam	165	1

Orders_Norm

order_id	order_item	order_amount	customer_id
empty			

Shippings

shipping_id	status	customer